

Virtual Desktop Pet & AI Assistant

Domain : Open Innovation

Team : CODING ROCKERZ

- Anmay Sunil Nayak
- Mohammed Nabeel Shaikh
- Ananya Bhat
- Amrutha Pai



HACK
MATRI
X

THE PROBLEM

Reminder apps are plain, generic utilities with no personality or engagement

They're completely disconnected from what user is actually doing on screen.

Users get no contextual help – its just notification then silence.

Who is affected?

Desktop power users & Remote workers → Teams relying on individual productivity → Workplace focus & Task-completion culture



WHY THIS PROBLEM EXISTS

Reminder
tools built as
isolated
localities

Privacy
concerns
discourage
screen-aware
features

AI assistants
live in browser
tabs not in
desktop



No Context
Awareness



Re-explains context
each time



Lost focus

Current Approach:

Manual Notes → App Switching → Delay → Missed deadlines



OUR SOLUTION

INTRODUCING Virtual Desktop Pet & AI Assistant

Enables desktop users to stay focused & get contextual help by pairing a companion pet with privacy first & screen-aware AI assistant

USER

↓
Question or reminder request
↓
Virtual Desktop pet & AI Assistant
↓
AI reads screen context + question
↓
Chat answer / Reminder alert
↓
Stays on track , less context switching

Core capabilities

Capability Value

01 [smart reminder] [recurring alert with pet animation ,so alerts feel alive]

02 [Contextual AI assistant] [ask about your screen ,get real answer]

03 [Privacy-first design] [Capture only on consent,always visibly active]



PROTOTYPE / PRODUCT EXPERIENCE

Prototype Screenshots 1

User journey

01 — INPUT

User ask pet a question, optionally selecting a window/screen to share.

↓

02 — PROCESS

AI analyzes the captured screen context alongside the question

↓

03 — OUTPUT

Answer appear in the pet's chat bubble

↓

04 — ACTION

User acts on the answer-or later, the pet fires the reminder with its alert animation.

Prototype Screenshots 2



WHAT MAKES US DIFFERENT?

WHY THIS APPROACH?

Existing Approach

Reminders are isolated from screen context
No AI assistance built in
Generic non customizable interface
No visible privacy indicators during data use
API keys often stored in plaintext/cloud synced

Virtual Desktop pet & AI Assistant

✓
✓
✓
✓
✓



TECHNICAL ARCHITECTURE

Tech stack:

Frontend :

Flutter(windows desktop)

Riverpod(state management)

Rive/Lottie(pet animations)

Window_manager,tray_manager

Backend :

Dart isolates/Timer

hotkey_manager,screen_capturer,dio(hot key,capture,AI API calls)

Storage :

Isar/Hive

flutter_secure_storage

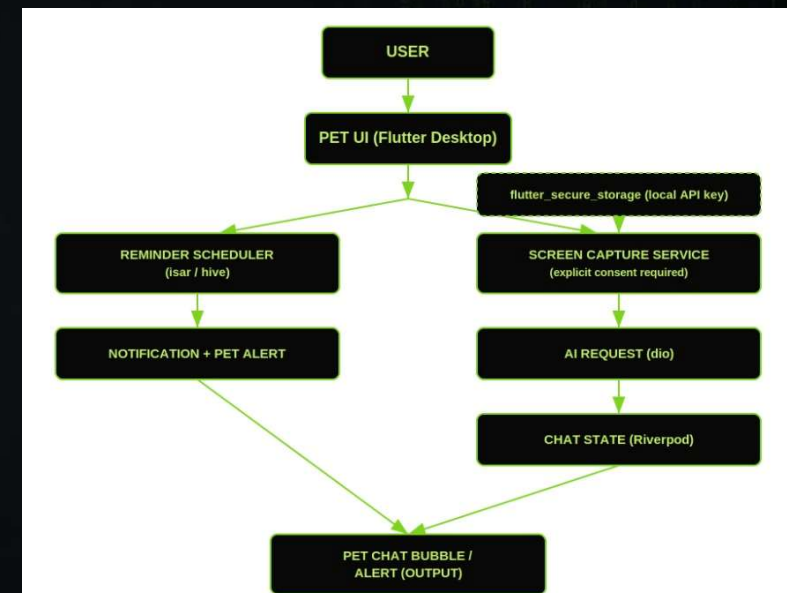
Deployment :

Flutter build windows

MSIX/Inno setup installer

NO cloud server – fully local +direct AI API calls

Workflow Image



FEASIBILITY + IMPACT

FEASIBILITY

Technical

1.Flutter-desktop – production ready

2.Screen capturer, isar, flutter-secure-storage

3.BYO AI API key via dio

Operational

DEV-machines+personal API keys

Local installer , no servers

Hackathon MVP

PET UI + Animation

Reminder CRUD +notification

AI screen+ Q&A with consent

IMPACT

Who benefits?

Desktop users, students, remote workers

What improves?

Faster counter recovery

Fewer missed reminders

Why does it matter?

Less cognitive overhead-less app switching

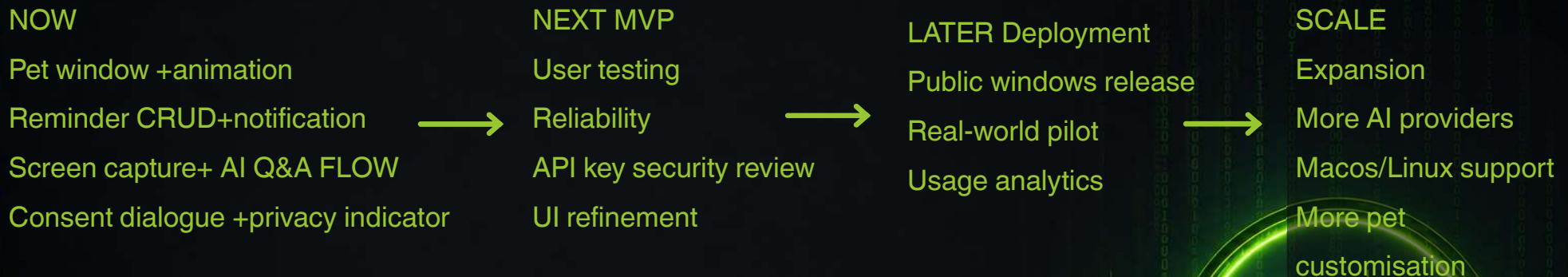
Then a bottom progression:

Prototype → MVP → Pilot → Scale



IMPLEMENTATION ROADMAP

WHAT HAPPENS NEXT?



Critical point Clearly separate:

Already built->the Now columns (pet window+ animations ,Reminder CRUD +notifications,screen capture +AI Q&A flow ,consent dialog +privacy indicator) vs.

Planned-> NEXT ,LATER, and SCALE columns

TEAM + EXECUTION



Anmay Nayak
Desktop window and
Backend engineer



Mohammed Nabeel Shaikh
AI Integration Engineer



Ananya Bhat
Testing and
documentaion



Amrutha Pai
UI/UX

